

University Appointment System - Full Documentation

This document contains the complete technical documentation for the University Appointment System, including system architecture, database models, API references, frontend structure, and deployment instructions.

1. System Architecture

The University Appointment System is built using a modern full-stack MERN-like architecture with a cross-platform mobile frontend.

- **Frontend:** Flutter (Dart). Compiles to Android, iOS, and Web.
- **Backend:** Node.js with Express.js.
- **Database:** MongoDB (via Mongoose).
- **Authentication:** JSON Web Tokens (JWT) for secure session management.
- **Real-time & Push:** Firebase Admin SDK on the backend, Firebase Messaging on the frontend.
- **Storage:** Local file system via Multer for document and image uploads.
- **Video Calls:** Jitsi Meet integration for telemedicine/virtual appointments.

2. Database Schema (Models)

The database consists of the following primary collections in MongoDB:

User (User.js)

Handles authentication and roles.

- name (String, required)
- email (String, required, unique)
- password (String, required)
- role (Enum: user, provider, admin, default: user)
- profilePicture (String)
- workHours (Array of objects containing day, startTime, endTime, isDayOff)
- pushToken (String, for notifications)
- pushNotificationsEnabled (Boolean)
- document (String, for provider verification)
- isVerified (Boolean, default: false)

Appointment (Appointment.js)

Tracks bookings between students and providers.

- userId (ObjectId, ref: User)
- serviceId (ObjectId, ref: Service)
- providerId (ObjectId, ref: User)
- date (Date)
- status (Enum: pending, approved, cancelled, completed, default: pending)
- notes (String)
- meetLink (String, auto-generated for virtual appointments)
- qrCodeData (String, for in-person check-in)

Service (Service.js)

Represents the offerings created by providers or admins.

- name (String, required)
- description (String, required)
- providerId (ObjectId, ref: User)
- duration (Number, in minutes)
- isVirtual (Boolean, default: false)

Message (Message.js)

Handles in-app chat regarding an appointment.

- appointmentId (ObjectId, ref: Appointment)
- senderId (ObjectId, ref: User)
- receiverId (ObjectId, ref: User)
- content (String, required)
- timestamp (Date, default: Date.now)

Notification (Notification.js)

Stores historical push notifications and in-app alerts.

- userId (ObjectId, ref: User)
- title (String)
- message (String)
- isRead (Boolean, default: false)
- type (Enum: info, success, warning, error)
- createdAt (Date)

Review (Review.js)

Feedback left by students after a completed appointment.

- appointmentId (ObjectId, ref: Appointment)
- userId (ObjectId, ref: User)
- providerId (ObjectId, ref: User)
- rating (Number, 1-5)
- comment (String)
- createdAt (Date)

3. API Reference

All endpoints are prefixed with `/api`. Most endpoints require a Bearer JWT token in the Authorization header.

Authentication (/api/auth)

- POST `/register`: Register a new user or provider.
- POST `/login`: Authenticate and receive a JWT.
- GET `/me`: Get current authenticated user profile.
- PUT `/profile`: Update profile info and preferences.

- PUT /work-hours: (Provider only) Update availability.

Services (/api/services)

- POST /: (Provider/Admin) Create a new service.
- GET /: List all available services.
- GET /provider/:id: List services offered by a specific provider.
- PUT /:id: (Provider/Admin) Update a service.
- DELETE /:id: (Provider/Admin) Remove a service.

Appointments (/api/appointments)

- POST /: Book a new appointment.
- GET /my: Get appointments for the logged-in user.
- GET /provider: (Provider) Get appointments assigned to the provider.
- PUT /:id/status: (Provider/Admin) Update appointment status (approve/cancel/complete).
- POST /emergency-absence: (Provider) Cancel all pending/approved appointments for a given day due to emergency.

Notifications (/api/notifications)

- GET /: Get all notifications for the current user.
- PUT /:id/read: Mark a specific notification as read.
- PUT /read-all: Mark all notifications as read.
- DELETE /:id: Delete a notification.

Reviews (/api/reviews)

- POST /: Submit a review for a completed appointment.
- GET /provider/:providerId: Get all reviews for a specific provider.

Messages (/api/messages)

- POST /: Send a message linked to an appointment.
- GET /:appointmentId: Get message history for an appointment.

Admin (/api/admin)

- GET /dashboard-stats: Get global system statistics.
- GET /users: List all users (with filtering for providers vs students).
- PUT /users/:id/role: Change user role.
- PUT /users/:id/verify: Verify a provider account.

4. Frontend Architecture

The Flutter application (frontend/lib) is structured by domain feature:

- screens/: Contains the UI layout pages.
 - auth/: Login and registration flow.
 - admin/: Super Admin dashboards (User management, system oversight).
 - provider/: Provider dashboards (Calendar, Service management, Work Hours, Analytics, QR Scanner).
 - user_dashboard.dart: Main student view for booking and managing appointments.
- providers/: State management classes utilizing the provider package to handle logic for AuthProvider, AppointmentProvider, ServiceProvider, etc.
- services/: Low-level classes managing HTTP requests (api_service.dart) and Firebase integration.
- widgets/: Reusable UI components like custom buttons, inputs, and cards.
- models/: Dart data classes representing the JSON payloads from the backend.

5. Deployment Guide

Environment Variables

Backend (.env)

```
PORT=5000
MONGO_URI=mongodb+srv://<user>:<password>@cluster.mongodb.net/appointments
JWT_SECRET=your_jwt_secret_key
```

Local Development

1. Backend:

```
cd backend
npm install
npm run dev
```

2. Frontend:

```
cd frontend
flutter pub get
flutter run
```

Production Deployment

Backend (e.g., Render, Heroku, DigitalOcean):

1. Set the Node.js environment to production.
2. Ensure MongoDB URI points to a production Atlas cluster.
3. Start command: node index.js.
4. *Note on Static Files:* The backend is configured to serve the Flutter Web build from the backend/public folder.

Frontend (Web):

1. Build the web app: flutter build web
2. Copy the contents of frontend/build/web to backend/public.
3. When the backend starts, it will serve the Flutter web app natively.

Frontend (Android):

1. Generate APK: flutter build apk --release
2. The APK can be distributed directly to students and providers.